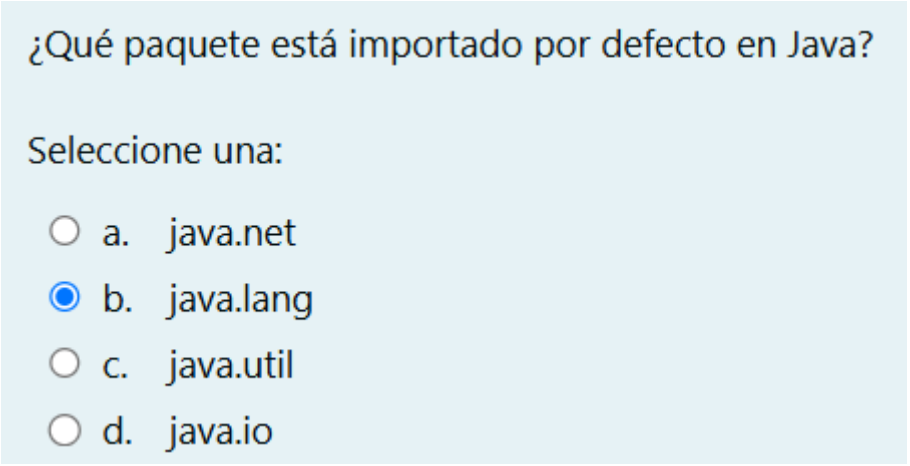
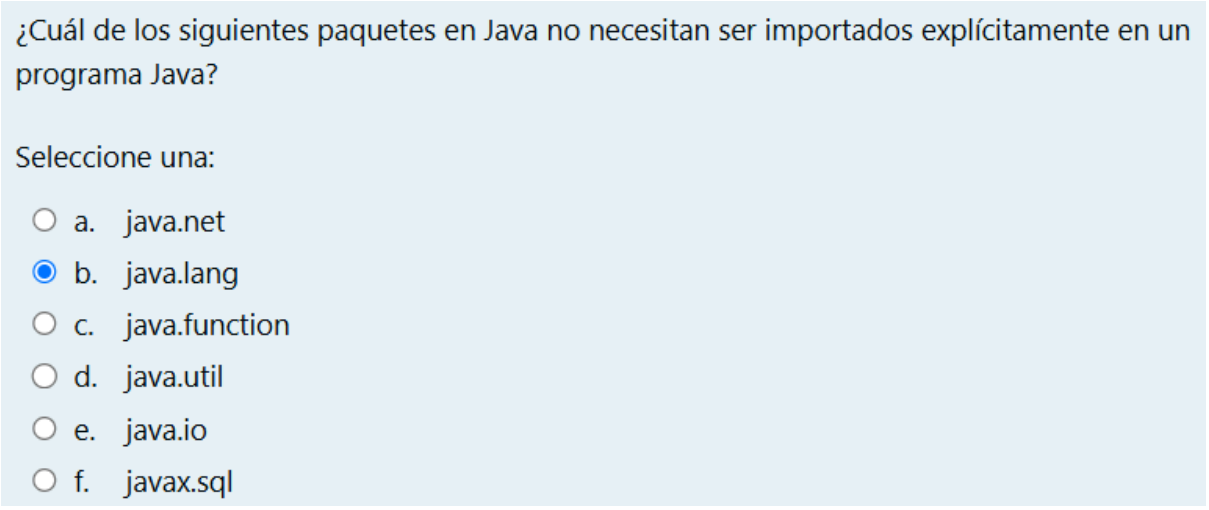
06 - Modificadores, static e inicialización

Pregunta número 1



Respuesta: En Java, el paquete `java.lang` se importa de manera automática y por defecto en absolutamente todas las clases. El lenguaje fue diseñado de esta forma porque dicho paquete contiene las clases base más esenciales y de uso constante, tales como String, System (la cual usas para System.out.println), Object (la superclase principal de todas las clases), Math y las clases envoltorio (Integer, Double, Boolean, etc.). Gracias a esta importación implícita, no es necesario que escribas import java.lang.*` al inicio de tus archivos para poder utilizarlas.

Pregunta número 2



Respuesta: El compilador de Java importa automáticamente el paquete java.lang en todo archivo fuente, como si cada clase tuviera implícitamente la línea import java.lang.*; por eso podemos usar String, System, Math, Object, Integer, Thread, Exception o StringBuilder sin importar nada. Básicamente, contiene las clases fundamentales del lenguaje, de las que depende prácticamente cualquier programa.

Pregunta número 3

¿Qué ocurre cuando ejecutas este fragmento de código?

```
class Utilidades {
    public static int sumar(int a, int b) {
        return a + b;
    }
}

public class TestStatic {
    public static void main(String[] args) {
        System.out.println("Resultado: " +
        Utilidades.sumar(5, 10));
    }
}
```

Seleccione una:

- a. Error de compilación porque sumar es estático.
- b. Imprime "Resultado: 15". (seleccionada)
- c. Error de ejecución.
- d. No se imprime nada.

Respuesta: El primer punto es que sumar es static, así que pertenece a la clase Utilidades y no a un objeto. Se invoca directamente con Utilidades.sumar(5, 10) sin crear una instancia. También debemos tener en cuenta que Utilidades no es public, pero está en el mismo archivo y, por tanto, en el mismo paquete que TestStatic. El acceso por defecto (package-private) basta. Finalmente, el orden de evaluación es que en “Resultado: “ + Utilidades.sumar(5, 10) primero se ejecuta la llamada al método, que devuelve el int 15. Luego se concatena con el String, dando “Resultado: 15”.

Pregunta número 4

What is a static block of code in Java?

Seleccione una:

- a. A block of code within a class that runs whenever the class is load on the JVM.
- b. A block of code inside a class that runs when that class first loaded in JVM
- c. A block of code within a class that always runs before the builder.

Respuesta: En Java, un bloque estático (static block) es un fragmento de código que se utiliza para inicializar variables estáticas. Su característica principal es su momento de ejecución: Primero se ejecuta exactamente una vez en toda la vida del programa. Lo segundo es que se ejecuta en el momento en que la clase es cargada por primera vez en la Máquina Virtual de Java (JVM), es decir, antes de que se pueda crear cualquier instancia de la clase o acceder a cualquier miembro estático.

Pregunta número 5

We have the following Java class:

```
package gal.dicoruna.example;
public class C {
    protected String v;
    ...
}
```

Seleccione una:

- a. From the class, the package, subclasses.
- b. Any site but only read if it is outside the package.
- c. From the class, the package, subclasses and all sites.

Respuesta: Debemos saber que el modificador de acceso `protected` nos permite acceder al miembro desde la propia clase, en este caso, la clase C; también nos permite acceder desde cualquier clase del mismo paquete (gal.dicoruna.example), sea o no una subclase, y finalmente, desde las subclases de C, aunque estén en otro paquete. Por lo tanto, la respuesta correcta es que podemos acceder desde la clase, el paquete y las subclases.

Pregunta número 6

What two keywords may precede the word 'class' in a class declaration?

Seleccione una o más de una:

- [ ] a. Synchronized
- [ ] b. Volatile
- [x] c. Static
- [ ] d. Local
- [x] e. Public

Respuesta: Siguiendo la sintaxis de Java, tenemos que la primera palabra sería el modificador de acceso `public`, que es el modificador que se utiliza para indicar que la clase es accesible desde cualquier otra clase en cualquier paquete. La siguiente palabra es static, que se utiliza específicamente para declarar clases anidadas. Una clase estática anidada no necesita una instancia de la clase externa para ser instanciada. Básicamente, declarar una clase interna como `static` indica que pertenece a la clase externa en sí y puede ser instanciada sin necesidad de crear un objeto de dicha clase externa.

Pregunta número 7

¿Cuáles son los modificadores que pueden preceder a una declaración de clase de nivel superior?

Seleccione una o más de una:

- [ ] a. protected
- [x] b. abstract
- [x] c. strictfp
- [ ] d. static
- [ ] e. private
- [x] f. public
- [ ] g. All the options are correct.
- [x] h. final

Respuesta: Una clase de nivel superior (top-level class) es aquella que se declara directamente dentro de un archivo fuente de Java, es decir, que no está anidada

dentro de otra clase. Ahora bien, los modificadores que pueden preceder a una clase de nivel superior son: El modificador de acceso public, con este modificador la clase es accesible desde cualquier otra clase en cualquier paquete. De ahí pasamos al abstract, que indica que la clase no puede ser instanciada directamente y está diseñada para ser extendida por otras clases. Seguimos con `final`, que también es válido e indica que la clase no puede ser heredada, o sea, que no puede tener subclases. Por último, tenemos `strictfp`, que asegura que los cálculos de punto flotante sean portables entre plataformas. Un punto a destacar es que a partir de Java 17 ya es redundante usar `strictfp` porque todos los cálculos son estrictos por defecto; sigue siendo sintácticamente válido en versiones anteriores a Java 17.

Pregunta número 8

What changes will make this code compile?

```
class X {
    X() { }
    private void one() { }
}
public class Y extends X {
    Y() { }
    private void two() {
        one();
    }
    public static void main(String[] args) {
        new Y().two();
    }
}
```

Seleccione una:

- a. Adding the public modifier to the declaration of class X.
- b. Adding the protected modifier to the X() constructor.
- c. Changing the private modifier on the declaration of the one() method to protected.
- d. Removing the Y() constructor.
- e. Removing the private modifier from the two() method.

Respuesta: El error de compilación en el código ocurre en la clase Y, específicamente dentro del método two(), al intentar ejecutar la llamada one(). En la clase X, el método one() tiene el modificador de acceso private. En Java, los miembros privados son estrictamente accesibles solo desde el interior de la propia clase donde fueron declarados. Aunque la clase Y hereda de la clase X (extends X), una subclase no tiene acceso directo a los métodos o campos privados de su superclase. Al cambiar el modificador de one() a protected, el método se vuelve accesible para cualquier subclase (como Y), así como para cualquier otra clase

dentro del mismo paquete. Esto hace que la llamada one(); sea válida y el código compile con éxito.

Pregunta número 9

Which statement is true?

```
class ClassA {
    public int numberOfInstances;
    protected ClassA(int numberOfInstances) {
        this.numberOfInstances = numberOfInstances;
    }
}
public class ExtendedA extends ClassA {
    private ExtendedA(int numberOfInstances) {
        super(numberOfInstances);
    }
    public static void main(String[] args) {
        ExtendedA ext = new ExtendedA(420);
        System.out.print(ext.numberOfInstances);
    }
}
```

Seleccione una:

- a. 420 is the output.
- b. An exception is thrown at runtime.
- c. All constructors must be declared public.
- d. Constructors CANNOT use the private modifier.
- e. Constructors CANNOT use the protected modifier.

Respuesta: El código está estructurado correctamente, compila sin errores y se ejecuta con éxito para mostrar el valor 420. Vamos a analizar el flujo para entender exactamente qué está sucediendo: ClassA define un constructor protected. Esto es completamente válido y permite que la clase hija ExtendedA lo invoque a través de la llamada super(numberOfInstances). ExtendedA define un constructor private. En Java, un constructor privado es legal y simplemente restringe la creación de objetos para que solo pueda realizarse desde dentro de la misma clase (una técnica común en el patrón de diseño Singleton o en clases utilitarias). Dado que el método main está declarado dentro de la misma clase ExtendedA, tiene visibilidad directa para invocar el constructor privado mediante new ExtendedA(420). Ese valor 420 fluye hacia arriba hasta el constructor de ClassA, se asigna a la variable pública numberOfInstances y, finalmente, el método System.out.print accede a ella y la imprime en consola.

Pregunta número 10

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Prioridad {
    {
        System.out.println("default");
    }
    static String saludo = "Variable";
    public Prioridad() {
        System.out.println("Constructor");
    }
    static {
        System.out.println("static");
    }
    public void getSaludo() {
        System.out.println(saludo);
    }
    public static void main(String[] args) {
        Prioridad p = new Prioridad();
        p.getSaludo();
    }
}
```

Seleccione una:

- a. Constructor Variable
- b. static default Constructor Variable
- c. static constructor default Variable
- d. Constructor static Variable

Respuesta: En el resultado del código podemos ver el orden estricto de inicialización que sigue Java al cargar una clase y crear una instancia de la misma. El primer flujo de ejecución nos muestra los bloques y variables estáticas: Se inicializan y ejecutan primero, una única vez, cuando la clase es cargada en memoria por la JVM (antes de entrar al flujo del main). El bloque static { System.out.println("static"); } imprime la palabra static. Seguimos con los bloques de

inicialización de instancia: Se ejecutan cada vez que se crea un nuevo objeto con la palabra reservada `new`, y ocurren estrictamente después del llamado a `super()` que se encuentra implícito aquí, pero antes del cuerpo del constructor. El bloque no etiquetado { System.out.println("default"); } se ejecuta e imprime default. Constructor: Se ejecuta inmediatamente después de los bloques de inicialización de instancia de la clase actual. El constructor public Prioridad() imprime la palabra Constructor. De ahí nos vamos a la llamada a métodos: Una vez que el objeto p está completamente inicializado en el main, se ejecuta p.getSaludo(). Este método imprime el valor de la variable de clase saludo, mostrando finalmente “Variable” en la consola.

Pregunta número 11

¿Cuál es el resultado al ejecutar el siguiente código?

```
class StaticNonStaticBlockTest {
    static {
        System.out.println("Static block");
    }
    {
        System.out.println("Instance block");
    }
    StaticNonStaticBlockTest() {
        System.out.println("Constructor");
    }
    public static void staticMethod() {
        System.out.println("Static method");
    }
    public static void main(String[] args) {
        StaticNonStaticBlockTest.staticMethod();
        new StaticNonStaticBlockTest();
    }
}
```

Seleccione una:

- a. Static method Static block Instance block Constructor
- b. Static block Instance block Constructor Static method
- c. Static block Static method Instance block Constructor

Respuesta: El resultado nos muestra cómo Java maneja el orden de ejecución al combinar la carga de la clase, llamadas a métodos estáticos y la creación de objetos. Veamos el flujo paso a paso para entender mejor: Primero tenemos la carga de la clase (bloque estático): Antes de que el método main comience a ejecutarse, la JVM carga la clase en memoria. Durante esta fase de carga, se ejecuta el bloque static una única vez, imprimiendo "Static block". Continuamos con la llamada al método estático: Al entrar al método main, la primera instrucción es StaticNonStaticBlockTest.staticMethod(); esto invoca el método directamente, lo que imprime "Static method". Seguimos con la creación del objeto (bloque de instancia): La segunda instrucción en el main es new StaticNonStaticBlockTest();, lo que inicia el proceso de instanciación. Antes de llegar al constructor, se ejecutan los bloques de inicialización de instancia (los bloques de código entre llaves {} sin la palabra static), imprimiendo "Instance block". Finalizamos con el constructor: Inmediatamente después del bloque de instancia, finaliza la creación del objeto ejecutando el cuerpo del constructor, imprimiendo "Constructor".

Pregunta número 12

What is the result?

```
public class Test {
    String value = "A";
    {
        value += "B";
    }
    {
        value += "C";
    }
    public Test() {
        value += "X";
    }
    public Test(String s) {
        value += s;
    }
    public static void main(String[] args) {
        Test m = new Test("M");
        m = new Test();
        System.out.println(m.value);
    }
}
```

Seleccione una:

- a. ABCXM
- b. ABCMABCB
- c. ABCM
- d. ABCX
- e. An exception is thrown
- f. The code does not compile

Respuesta: Este resultado se debe a que se instancian dos objetos completamente diferentes de la clase Test y la variable m se reasigna, por lo que solo se imprime el valor del segundo objeto. Tenemos que ver el flujo de ejecución paso a paso para entender mejor: Empezamos con la primera instanciación (Test m = new Test("M");), aquí se crea un nuevo objeto en memoria. Su variable de instancia value se inicializa explícitamente con "A". Se ejecutan los bloques de inicialización de instancia de arriba hacia abajo. El primero concatena "B" (valor actual: "AB") y el segundo concatena "C" (valor actual: "ABC"). Se ejecuta el constructor que recibe un String, concatenando el argumento "M". El valor final de value para este primer objeto es "ABCM". Vamos con la segunda instanciación y reasignación (m = new Test();): Se crea un nuevo objeto, reiniciando el ciclo de vida. Su variable value inicia en "A". Se ejecutan nuevamente los bloques de instancia secuencialmente, concatenando "B" y "C" (valor actual: "ABC"). Se ejecuta el constructor sin argumentos (Test()), el cual concatena "X". El valor final de value para este segundo objeto es "ABCX". La referencia m se actualiza para apuntar a este nuevo objeto (El primer objeto con valor "ABCM" queda suelto en memoria y elegible para el recolector de basura). Finalizamos con la impresión (System.out.println(m.value);): Imprime el valor de la variable del objeto que actualmente sostiene la referencia m, mostrando "ABCX" en la consola.

Pregunta número 13

¿Cuál será el resultado de compilar y ejecutar el siguiente fragmento de código en Java?

```
public class StaticBlockTest {
    static {
        System.out.println("Static block 1");
    }
    {
        System.out.println("Instance block 1");
    }
    public StaticBlockTest() {
        System.out.println("Constructor");
    }
    static {
        System.out.println("Static block 2");
    }
    {
        System.out.println("Instance block 2");
    }
    public static void main(String[] args) {
        new StaticBlockTest();
        new StaticBlockTest();
    }
}
```

Seleccione una:

a.
```
Static block 1
Static block 2
Instance block 1
Constructor
Instance block 2
Instance block 1
Constructor
Instance block 2
```

b. Error de compilación.

c.
```
Static block 1
Static block 2
Instance block 1
Instance block 2
Constructor
Instance block 1
Instance block 2
Constructor
```

d.
```
Instance block 1
Instance block 2
Constructor
Instance block 1
Instance block 2
Constructor
Static block 1
Static block 2
```

Respuesta: En este código se nos demuestra las reglas de orden de inicialización en Java cuando se combinan múltiples bloques estáticos, bloques de instancia y constructores en una misma clase. Analicemos el flujo de ejecución para poder entender qué está pasando: Primero tenemos la carga de la clase (bloques

estáticos): Antes de que el método main comience a ejecutarse, la Máquina Virtual de Java carga la clase en memoria. En este momento, se ejecutan todos los bloques static en el orden estricto en que aparecen en el código fuente, y esto ocurre una sola vez. Por lo tanto, se imprime "Static block 1" seguido de "Static block 2". De ahí pasamos a la primera instanciación: Al llamar al primer new StaticBlockTest(); dentro del main, se inicia la creación del primer objeto. Primero se ejecutan todos los bloques de inicialización de instancia en el orden en que aparecen, imprimiendo "Instance block 1" y "Instance block 2". Inmediatamente después de estos bloques, se ejecuta el cuerpo del constructor, imprimiendo "Constructor". Continuamos con la segunda instanciación: La siguiente línea es un segundo new StaticBlockTest();. Los bloques estáticos ya no se ejecutan porque la clase ya fue cargada, pero el ciclo de vida de instancia se repite completo. Se vuelven a ejecutar los bloques de instancia ("Instance block 1", "Instance block 2") seguidos nuevamente por el constructor ("Constructor").